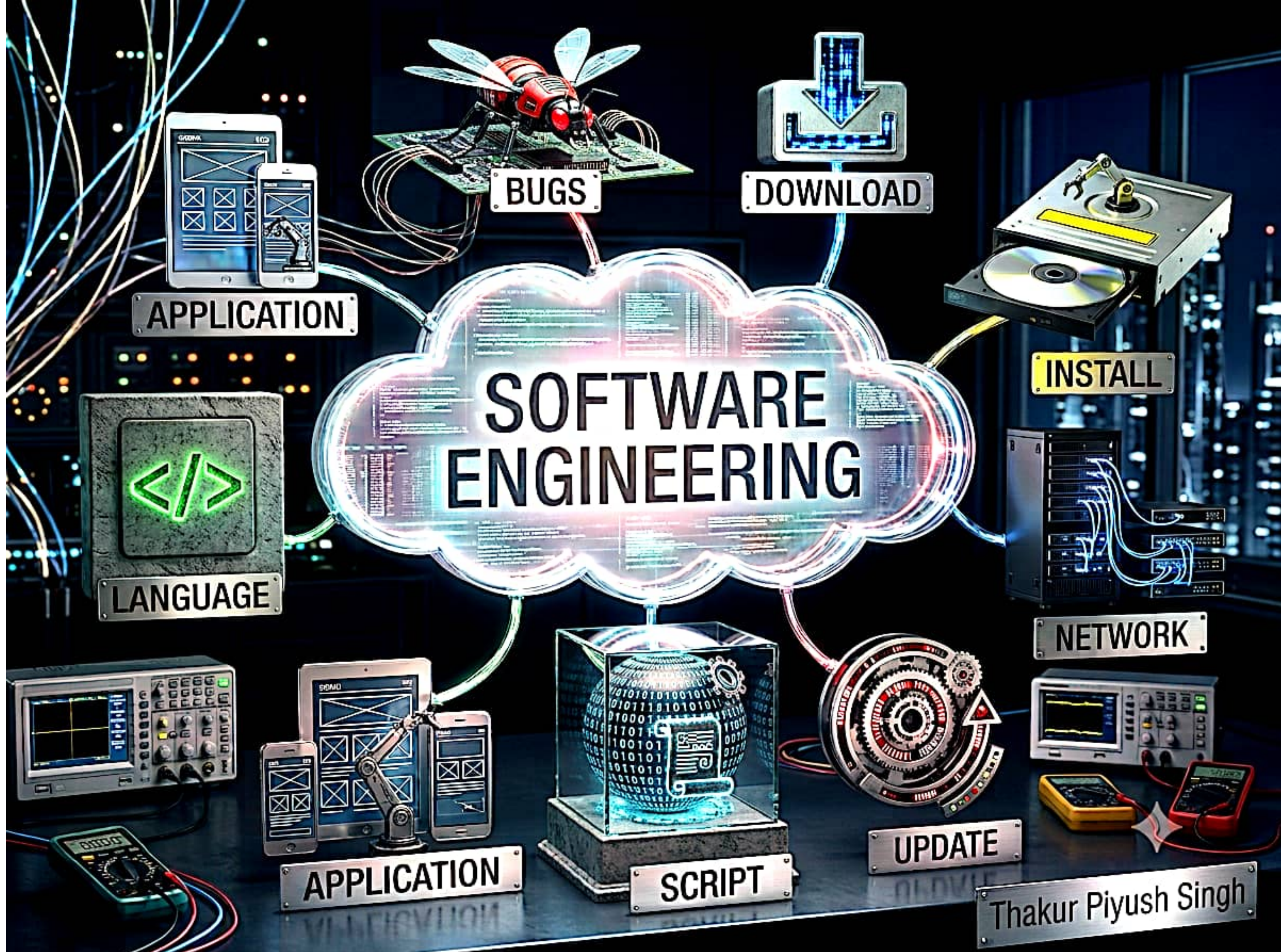
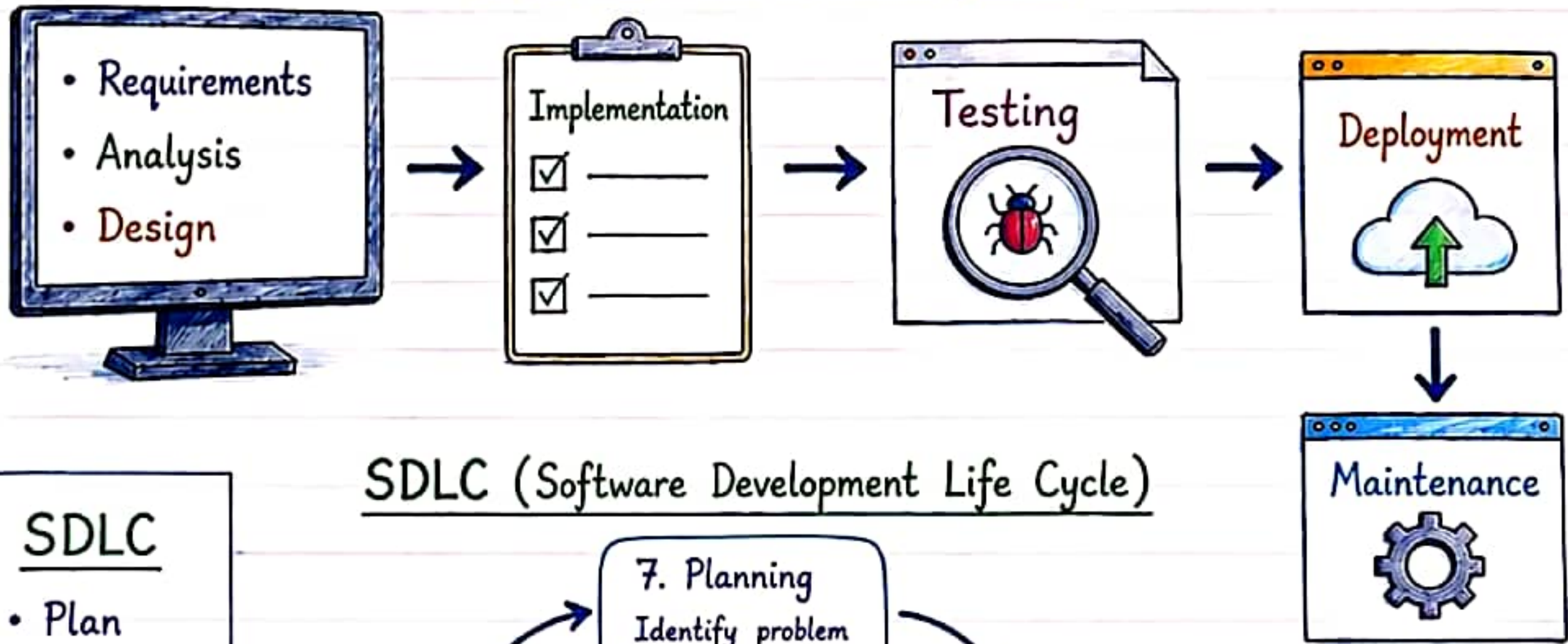




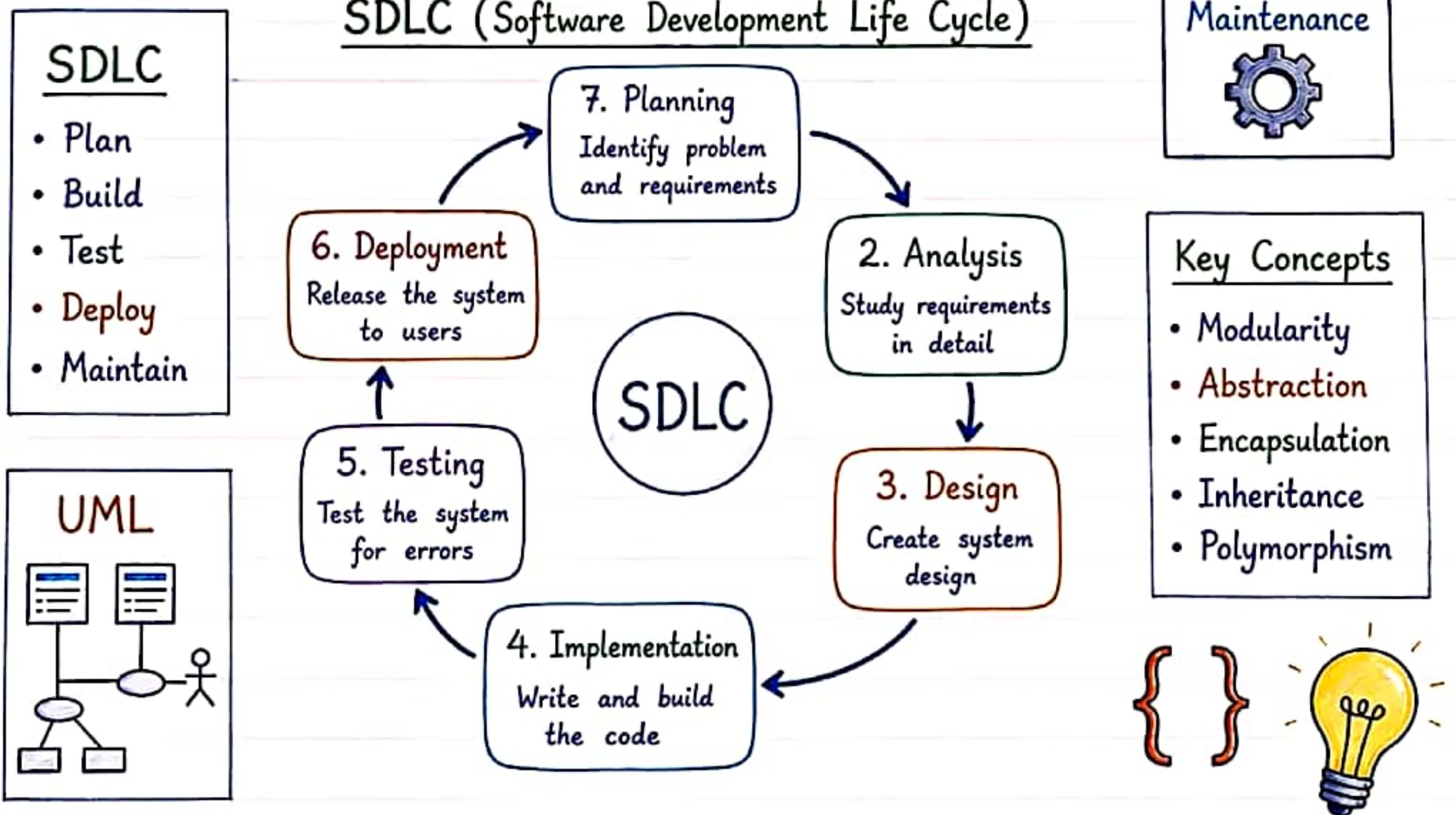
Thakur Piyush Singh

Software Engineering Notes – Part 1

@ Mastercode.sagar



SDLC (Software Development Life Cycle)



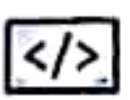
Requirements – What the system should do.



Analysis – Understanding the problem.



Design – Planning the solution.



Implementation – Writing the actual code.



Testing – Finding and fixing errors.



Deployment – Delivering the system to users.



Maintenance – Updating and improving the system.

Good software
is built with
good process!

Let's build software the right way!

@ Mastercode.sagar

Chapter 1: Introduction to Software Engineering

✓ 1. Definition of Software

Software is a collection of programs, data, and documentation that enables a computer to perform specific tasks.

→ Examples:



Windows



Chrome



WhatsApp

✓ 2. Software Engineering

Software Engineering is the systematic approach to designing, developing, testing, and maintaining software.

→ Objectives:



High-quality software



Low development cost



Timely delivery



Easy maintenance

✓ 3. Characteristics of Software

- | | |
|-----------------------------------|------------------------|
| ★ Developed, not manufactured | ★ Highly complex |
| ★ Does not wear out like hardware | ★ Easy to copy |
| ★ Intangible (cannot be touched) | ★ Requires maintenance |



✓ 4. Software Crisis

Software Crisis refers to problems faced in software development due to increasing complexity.



Causes:

- Poor planning
- Inaccurate cost estimation
- Lack of testing
- Poor documentation



Effects:

- Project delays
- High costs
- Low-quality software



Solution:

Use Software Engineering principles and SDLC models.

✓ 5. Software Myths



Customer Myths



- Requirements can be changed anytime easily.
- A brief idea is enough to start coding.



Management Myths



- Adding more programmers speeds up a delayed project.
- Existing standards alone ensure success.



Developer Myths



- Coding is the only important task.
- Software is complete once it works.

6: Software Development Process

Definition:

Software Development Process is a set of **activities** used to develop software systematically.

Benefits:

- ✓ Better planning
- ✓ Improved quality
- ✓ Reduced development cost
- ✓ Easier project management

Main Activities:

-  → 1. Requirement Gathering – Collect user needs.
-  → 2. System Design – Create software architecture.
-  → 3. Coding (Implementation) – Write program code.
-  → 4. Testing – Find and fix errors.
-  → 5. Deployment – Deliver software to users.
-  → 6. Maintenance – Update and improve software.

Key Point:



A structured process helps deliver reliable and efficient software.

7: Software Quality Attributes

Definition:

Software Quality Attributes are characteristics that determine the **quality** and **performance** of software.

Important Quality Attributes:

	1. Correctness	Software meets user requirements.
	2. Reliability	Software works without failure.
	3. Efficiency	Uses minimum resources and provides fast performance.
	4. Usability	Easy to learn and use.
	5. Maintainability	Easy to modify and fix.
	6. Portability	Can run on different platforms.
	7. Security	Protects data from unauthorized access.
	8. Scalability	Handles increasing workload effectively.



8. Software Applications

✓ Definition

Software applications are programs designed to perform specific tasks for users or organizations.

✓ Types of Software Applications

① System Software



→ Manages computer hardware and provides a platform for other software.

Examples:
Windows, Linux, Android

② Application Software



→ Performs specific user tasks.

Examples:
MS Word, Excel, WhatsApp

③ Web Applications



→ Run through web browsers using the internet.

Examples:
Gmail, YouTube, Facebook

④ Mobile Applications



→ Designed for smartphones and tablets.

Examples:
Instagram, Google Maps

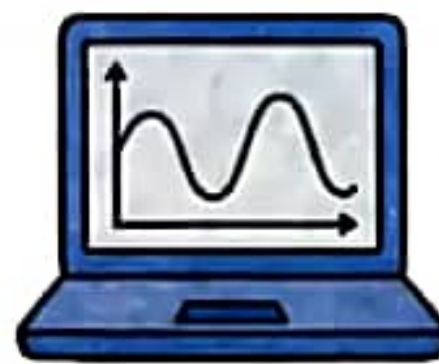
⑤ Business Software



→ Used for business operations and management.

Examples:
ERP Systems, Payroll Systems

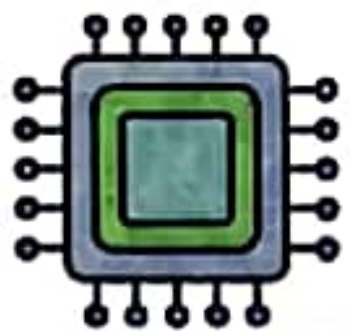
⑥ Scientific and Engineering Software



→ Used for research, simulations, and calculations.

Examples:
MATLAB, AutoCAD

⑦ Embedded Software



→ Installed within hardware devices to control operations.

Examples:
Washing Machines, Smart TVs, Cars

✓ Key Point



Automate
Tasks



Improve
Productivity



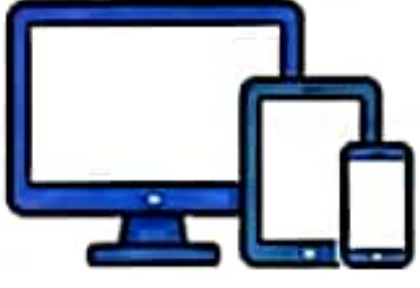
Solve Real-world
Problems

9. Software Quality Attributes

✓ Definition

Software Quality Attributes are characteristics that determine how well software performs and satisfies user requirements.

✓ Important Quality Attributes

- | | | | | |
|---|-----------------|---|--|---|
| ① | Correctness | → |  | Software should produce accurate and expected results. |
| ② | Reliability | → |  | Software should perform consistently without failures. |
| ③ | Efficiency | → |  | Software should use resources such as memory and CPU effectively. |
| ④ | Usability | → |  | Software should be easy to learn and use. |
| ⑤ | Maintainability | → |  | Software should be easy to modify and update. |
| ⑥ | Portability | → |  | Software should run on different platforms and operating systems. |
| ⑦ | Security | → |  | Software should protect data from unauthorized access. |
| ⑧ | Scalability | → |  | Software should handle increasing users and workload efficiently. |
| ⑨ | Availability | → |  | Software should remain accessible whenever needed. |

2.1 Software Development Life Cycle (SDLC)

Software Development Life Cycle (SDLC) is a systematic process used for planning, developing, testing, deploying, and maintaining software. It provides a structured approach that helps in producing high-quality software within time and budget constraints.



The main objective of SDLC is to develop reliable and efficient software that meets user requirements.

✓ Phases of SDLC

- 1

Requirement Gathering

→

In this phase, developers collect and analyze the requirements of users and stakeholders. The requirements are documented for future reference.
- 2

System Design

→

Based on the requirements, the software architecture, database structure, and system design are prepared.
- 3

Implementation (Coding)

→

Developers write program code according to the design specifications using appropriate programming languages.
- 4

Testing

→

The software is tested to identify and remove defects. Testing ensures that the software functions correctly.
- 5

Deployment

→

After successful testing, the software is delivered and installed for end users.
- 6

Maintenance

→

The software is updated and modified to fix bugs, improve performance, and meet changing user needs.



2.2 Waterfall Model

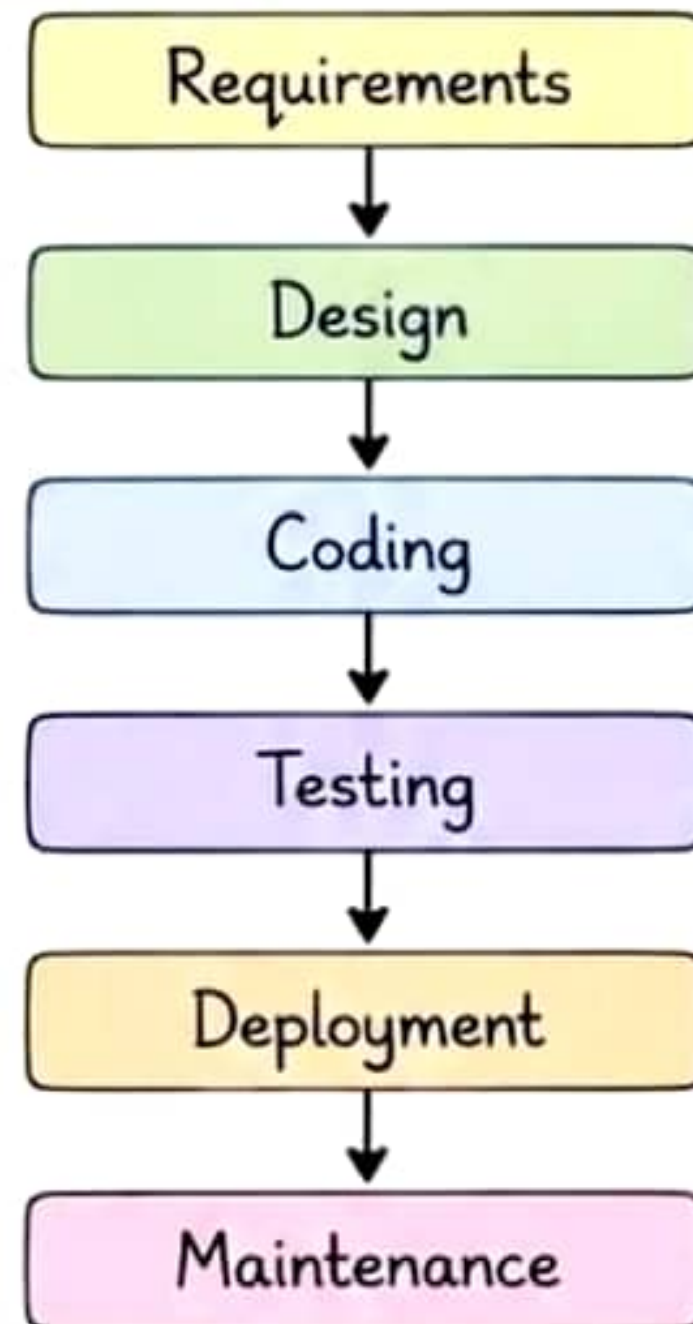
✓ Definition

The Waterfall Model is a **linear and sequential** SDLC model where each phase must be completed before moving to the next phase.

✓ Phases of Waterfall Model

- | | | |
|---|---|-------------------------|
| ① |  | Requirements Analysis |
| ② |  | System Design |
| ③ |  | Implementation (Coding) |
| ④ |  | Testing |
| ⑤ |  | Deployment |
| ⑥ |  | Maintenance |

✓ Diagram



✓ Advantages

- Simple and easy to understand
- Easy project management
- Well-defined phases
- Suitable for small projects
- Proper documentation

✓ Disadvantages

- Difficult to handle requirement changes
- Testing starts late
- High risk in large projects
- Customer feedback is limited

✓ Applications

- | | |
|--|---|
|  | ➔ Banking Systems |
|  | ➔ Payroll Systems |
|  | ➔ Inventory Management Systems |
|  | ➔ Government Projects with fixed requirements |



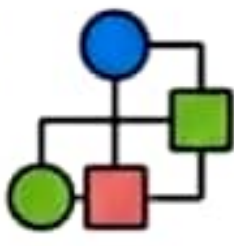
2.3 V-Model

✓ Definition

The V-Model (Verification and Validation Model) is an SDLC model where testing activities are planned alongside each development phase. Every development phase has a corresponding testing phase, ensuring early defect detection and better software quality.

✓ Phases of V-Model

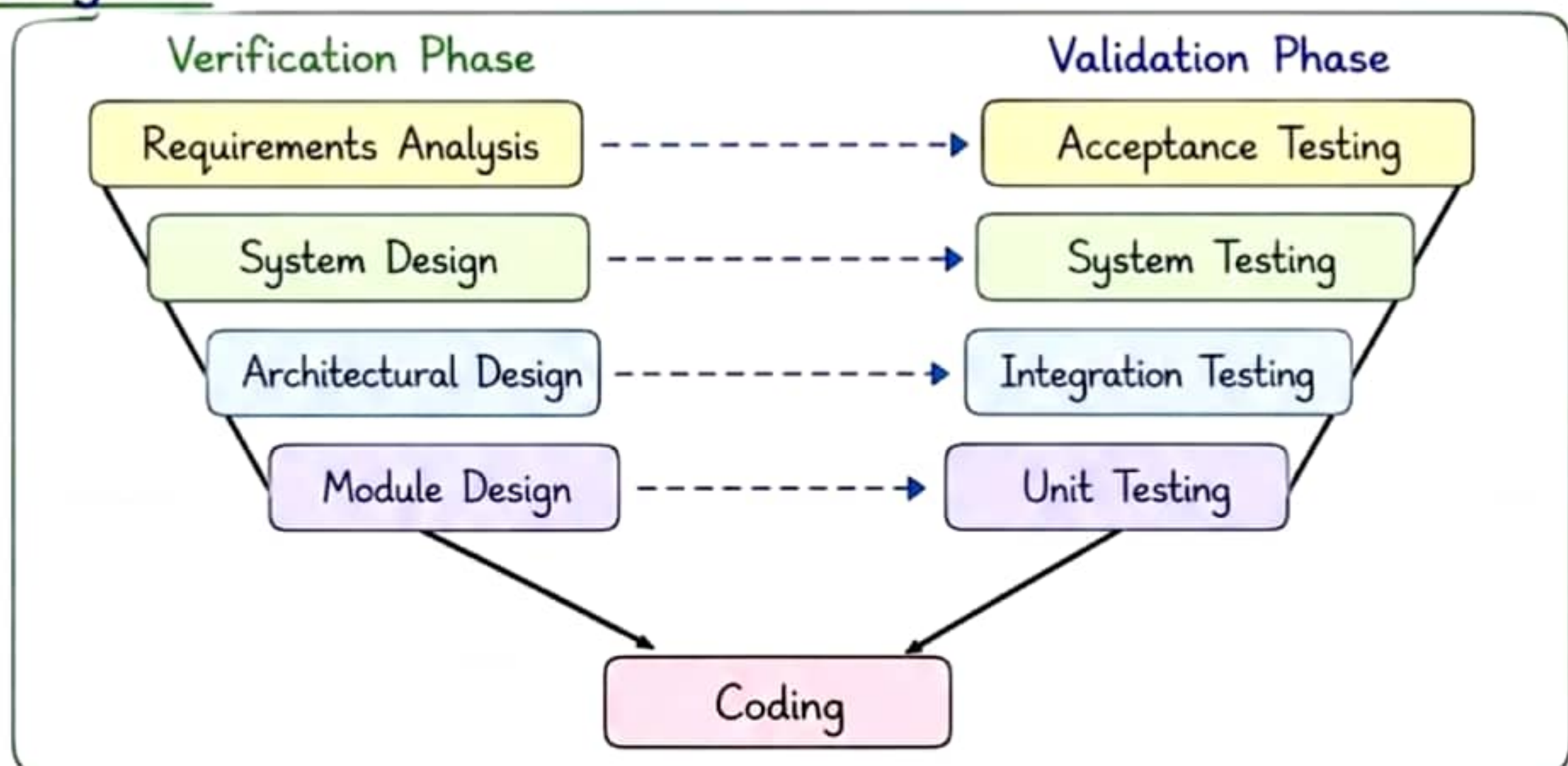
Verification Phase

- ①  **Requirements Analysis**
⇒ Gather and analyze user requirements.
- ②  **System Design**
⇒ Design the overall system architecture.
- ③  **Architectural Design**
⇒ Divide the system into modules and components.
- ④  **Module Design**
⇒ Create detailed design for individual modules.
- ⑤  **Coding**
⇒ Develop the software according to the design.

Validation Phase

- ①  **Acceptance Testing**
⇒ Verify software against customer requirements.
- ②  **System Testing**
⇒ Test the complete software system.
- ③  **Integration Testing**
⇒ Test interaction between modules.
- ④  **Unit Testing**
⇒ Test individual modules.

✓ Diagram



✓ Definition

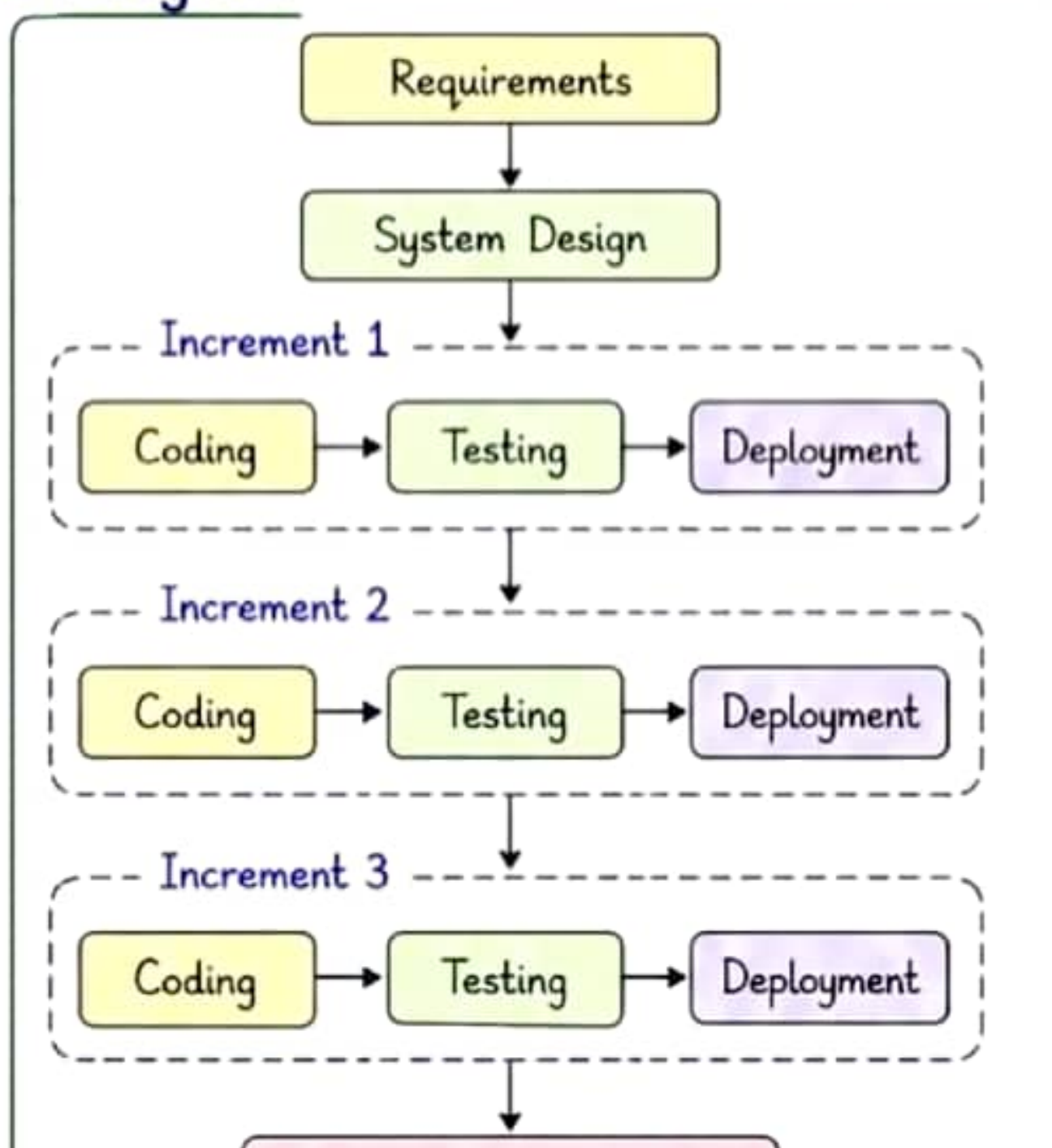
The Incremental Model is an SDLC model in which software is developed and delivered in small parts called **increments**. Each increment adds new features and functionality until the complete software is developed.

This model combines the advantages of both the Waterfall Model and Prototyping Model.

✓ Phases of Incremental Model

- 1**  **Requirement Analysis**
⇒ Gather and analyze user requirements.
⇒ Divide requirements into multiple increments.
- 2**  **System Design**
⇒ Design the overall software architecture.
- 3**  **Development of Increment 1**
⇒ Develop and test the first set of features.
- 4**  **Deployment of Increment 1**
⇒ Deliver the first working version to users.
- 5**  **Development of Increment 2**
⇒ Add additional features and functionalities.
- 6**  **Deployment of Increment 2**
⇒ Release the updated version.
- 7**  **Further Increments**
⇒ Continue development until all requirements are completed.
- 8**  **Final Product**
⇒ Complete software is delivered.

✓ Diagram



✓ Advantages

- Working software is delivered early.
- Easy to accommodate changes.
- Lower development risk.
- Faster customer feedback.
- Easier testing and debugging.
- Suitable for large projects.

✗ Disadvantages

- Requires good planning and design.
- Integration of increments can be complex.
- Higher management effort.
- Not suitable for very small projects.
- Architecture must be well defined from the beginning.

✓ Applications

- E-Commerce Websites
- Mobile Applications
- Banking Applications
- Web Applications

✓ Definition

The Spiral Model is a **risk-driven** SDLC model that combines the features of the Waterfall Model and Prototyping Model. Development proceeds through repeated cycles called spirals, with each cycle focusing on planning, risk analysis, development, and evaluation.

It is especially suitable for large, complex, and high-risk projects.

✓ Phases of Spiral Model

- 1**



Planning
 - ➔ Gather requirements.
 - ➔ Define project objectives and constraints.
- 2**



Risk Analysis
 - ➔ Identify and analyze potential risks.
 - ➔ Develop strategies to minimize risks.
- 3**



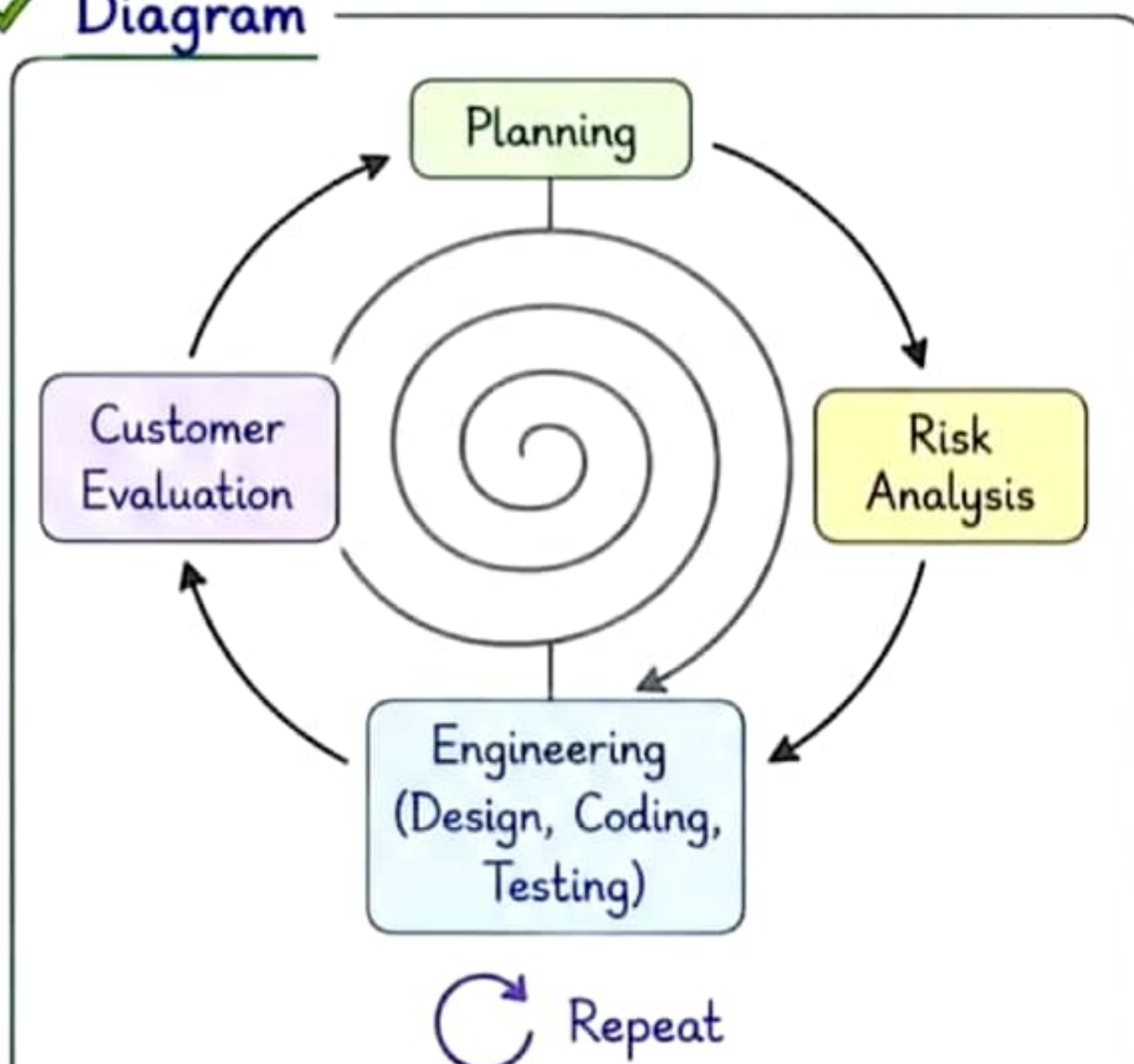
Engineering (Development)
 - ➔ Design, code, and test the software.
- 4**



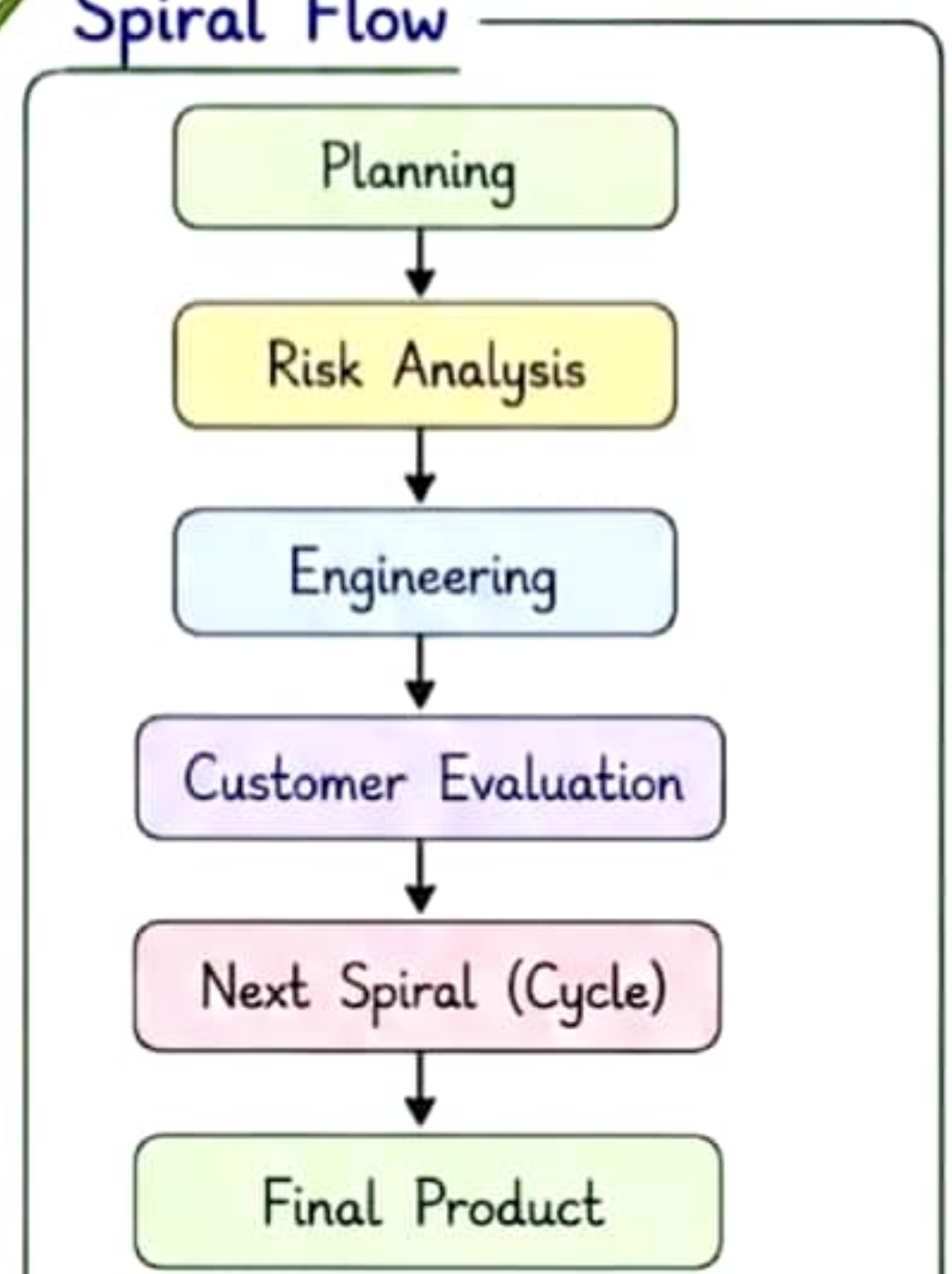
Customer Evaluation
 - ➔ Obtain customer feedback.
 - ➔ Review the developed version and plan the next cycle.

The process repeats until the final software product is completed.

✓ Diagram



✓ Spiral Flow



3.1 Requirements Engineering

✓ Definition

Requirements Engineering is the process of gathering, analyzing, documenting, validating, and managing the requirements of a software system.

It helps developers understand what the customer needs and ensures that the final software meets those needs.

Objectives



Understand customer requirements



Reduce development errors



Improve software quality



Ensure customer satisfaction

3.2 Functional Requirements

✓ Definition

Functional Requirements describe what the system should do. They specify the functions and services that the software must provide.

Examples



User login and logout



User registration



Online payment processing



Search functionality

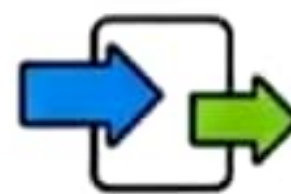


Report generation

Characteristics



Define system behavior



Describe inputs and outputs



Directly related to user requirements

✓ Definition

Non-Functional Requirements describe **how** the system should perform rather than **what** it should do.

Examples

-  Performance
-  Security
-  Reliability
-  Usability
-  Scalability

Characteristics



Define quality attributes



Improve user experience



Ensure system efficiency

3.4 Requirement Gathering Techniques

✓ Definition

Requirement Gathering is the process of **collecting information** from stakeholders to understand their needs and expectations.

Techniques

1

Interview



- Direct discussion with stakeholders.
- Helps gather detailed information.

2

Questionnaire



- A set of written questions distributed to users.
- Useful for collecting information from many people.

3

Observation



- Observe users while they perform tasks.
- Helps understand actual workflows.

4

Workshops



- Group discussions involving stakeholders and developers.
- Encourages collaboration.

5

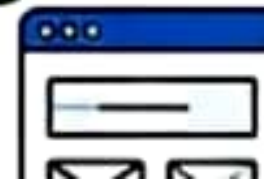
Brainstorming



- Generate ideas and solutions through group

6

Prototyping



- Create a sample model of the system.

✓ Definition

A Software Requirement Specification (SRS) is a formal document that describes all **functional** and **non-functional** requirements of a software system. It acts as an agreement between the customer and the development team.

✓ Contents of SRS

1



Introduction

- Purpose of the software
- Scope of the project

2



Functional Requirements

- Features and services of the system

3



Non-Functional Requirements

- Performance, security, reliability, etc.

4



System Constraints

- Hardware and software limitations

5



User Requirements

- Expectations and needs of users

✓ Benefits of SRS



Reduces
misunderstandings



Improves
communication



Provides a
development



Helps in testing
and maintenance

✓ Definition

Requirement Validation is the process of checking whether the documented requirements are **correct, complete, consistent, and feasible**.

Objectives



Ensure requirements meet customer needs



Detect errors early



Reduce development cost



Ensure requirements meet customer needs

✓ Validation Techniques

1



Reviews

→ Examine requirements documents.

2



Inspections

→ Detailed checking by experts.

3



Prototyping

→ Validate requirements through a working model.

4



Test Case Generation

→ Create test cases to verify requirements.

✓ Advantages of Requirements Engineering



Better understanding



Reduced project risk



Improved software



Lower development



Increased customer

4.1 System Design

✓ Definition

System Design is the process of defining the **architecture**, **components**, **modules**, **interfaces**, and **data** of a system to satisfy specified requirements. It acts as a blueprint for software development.

Objectives



- Convert requirements into design.



- Improve software quality.



- Reduce development complexity.



- Facilitate implementation and maintenance.

✓ Definition

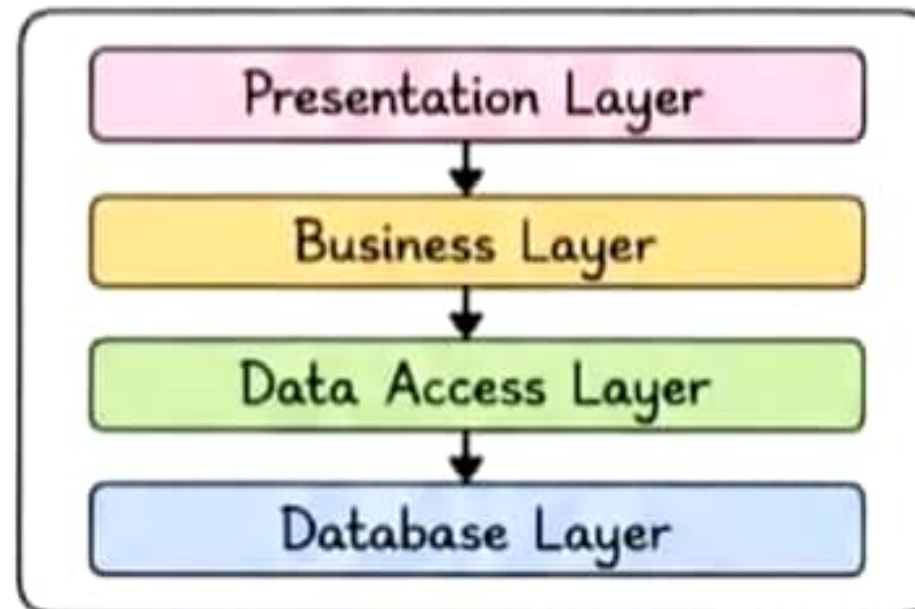


- Architectural Design defines the overall structure of the software system and the relationships between its components.
- It describes how different modules interact with each other.

✓ Types of Architecture

1

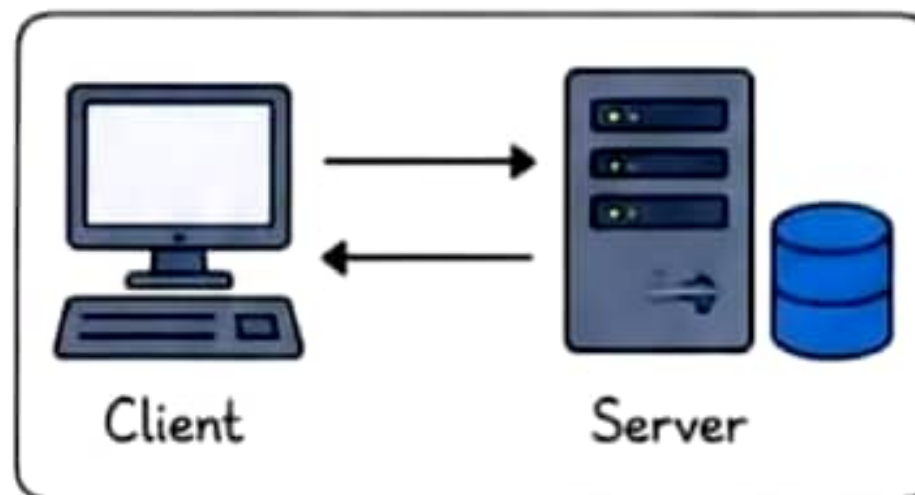
Layered Architecture



The system is divided into multiple layers. Each layer has a specific role and communicates with adjacent layers.

2

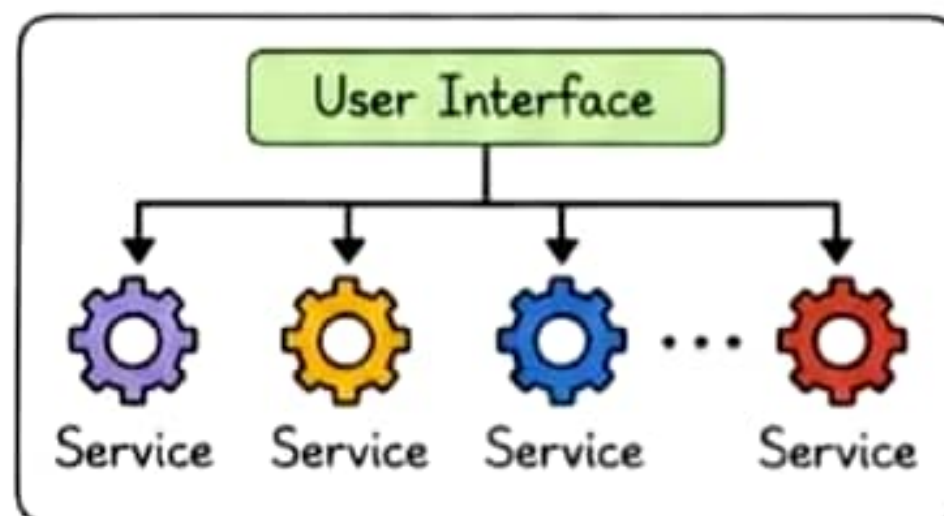
Client-Server Architecture



Clients send requests to the server and the server processes them and sends back the response.

3

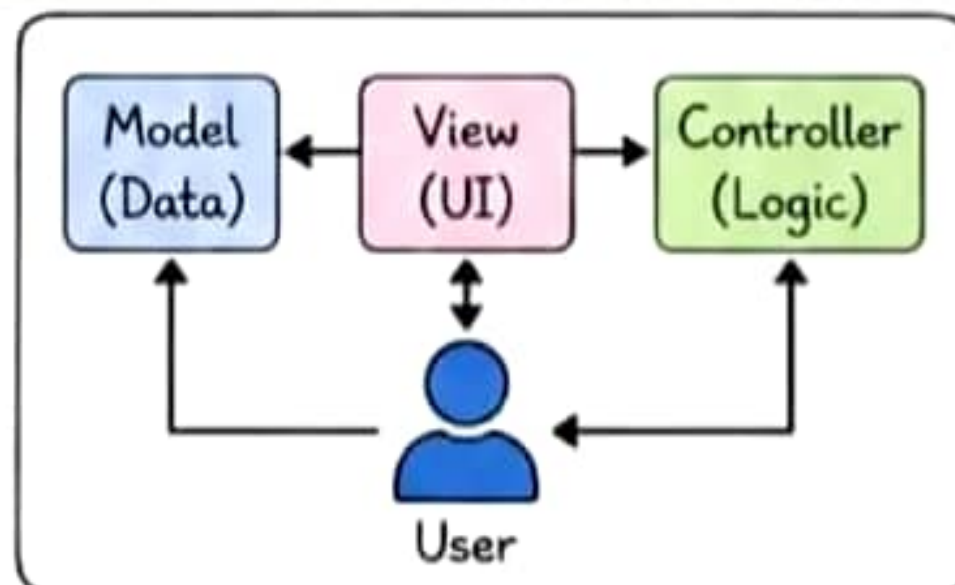
Microservices Architecture



The application is built as a collection of small, independent services that communicate with each other.

4

MVC Architecture



The application is separated into Model (data), View (UI), and Controller (logic) to improve flexibility and maintainability.

✓ Benefits



Better Scalability

The system can handle increasing users and



Improved Maintainability

The system is easier to update, modify, and



Easier System Integration

Different modules and components can be